

Week 06: Building & Training a Modern LLM

What Changed Since 2017 — and Why

WEEK

06

TOPIC

MODERN LLM ARCHITECTURE

DATE

21 FEB 2026

MODELS

LLAMA | MISTRAL | GEMMA

The original Transformer from 2017 ("Attention Is All You Need") revolutionized AI, but modern models like LLaMA, Mistral, Gemma, and DeepSeek don't use it as originally designed. Today, we dissect the four critical upgrades that make modern Open-Weight LLMs scalable, fast, and stable — and understand the mathematical "why" behind each one.

Quick Links & Resources

Resource	Link
 Lecture Slides	Google Drive
 Excalidraw Board	Interactive Whiteboard
 GQA Deep Dive	Syhya Blog: Comparing MHA, MQA, & GQA

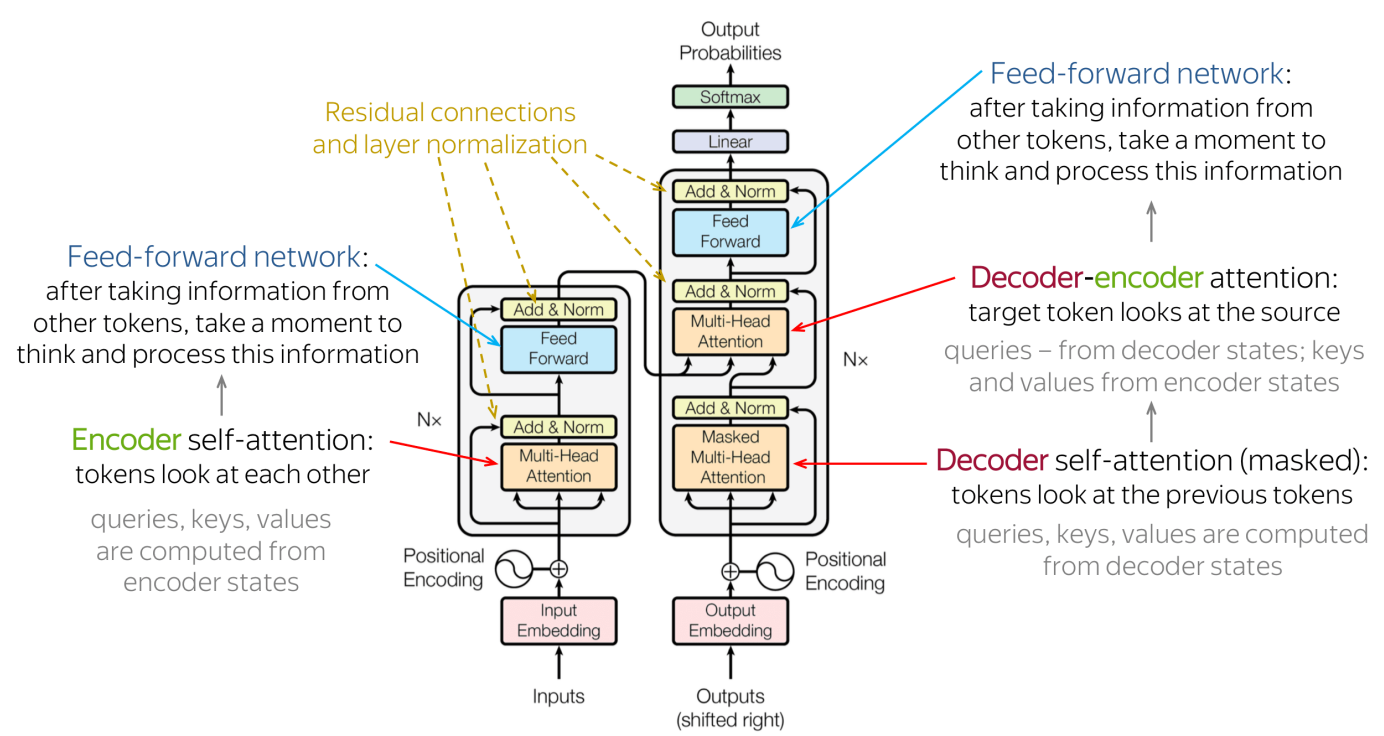
Table of Contents

Section	Topic	Description
1	The Classic Transformer	The Baseline: GPT-2 era architecture
2	The Four Upgrades	The component-by-component evolution
3	RMSNorm & Pre-Norm	Stabilizing 30+ layer deep networks
4	SwiGLU Activation	Learned gating for smarter information flow
5	RoPE	Relative distance via rotation
6	Attention: MHA → GQA	KV Cache, Time/Space Complexity & GQA
7	The Modern Block	All 4 upgrades assembled together
	Quick Revision Card	The entire lecture on one page

[1](#) The Classic Transformer: The 2017 Baseline

Before we can appreciate the modern LLM, we must understand what we're upgrading *from*. The original 2017 Transformer (and essentially GPT-2 from 2019) was built from four key components:

#	Component	What It Does
1	Token Embedding + Sinusoidal Position	Adds fixed, absolute wave patterns to token vectors to encode "I am at position #47."
2	LayerNorm (Post-Norm)	Normalizes the signal <i>after</i> each sublayer (Attention / FFN).
3	Multi-Head Attention (MHA)	Every attention head has its own dedicated Query, Key, and Value matrices.
4	FFN with ReLU	A simple Feed-Forward Network using $\max(0, x)$ as activation.



A step-by-step breakdown of the Classic Transformer architecture, showing the Encoder self-attention, Decoder masked self-attention, and Decoder-Encoder cross-attention interplay.

[!WARNING] **Why did we change it?** The Classic Transformer worked brilliantly for models up to ~1 Billion parameters. But when researchers tried to scale to **7B, 70B, or 400B+** parameters, three critical walls emerged:

- 1. Training Instability:** Gradients would explode or vanish across 30+ stacked layers (Post-Norm issue).
- 2. Memory Wall:** The KV Cache for standard MHA consumed 4–64 GB of VRAM per user, per request (MHA issue).
- 3. Computational Waste:** ReLU kills half of all activations with a blunt zero; sinusoidal positions can't generalize beyond their trained context length.

2 The Four Upgrades: Classic → Modern

Virtually every modern open-weight LLM (LLaMA, Mistral, Gemma, DeepSeek) has standardized on **four specific architectural swaps**. They are not random improvements; each one surgically addresses a specific scaling failure.

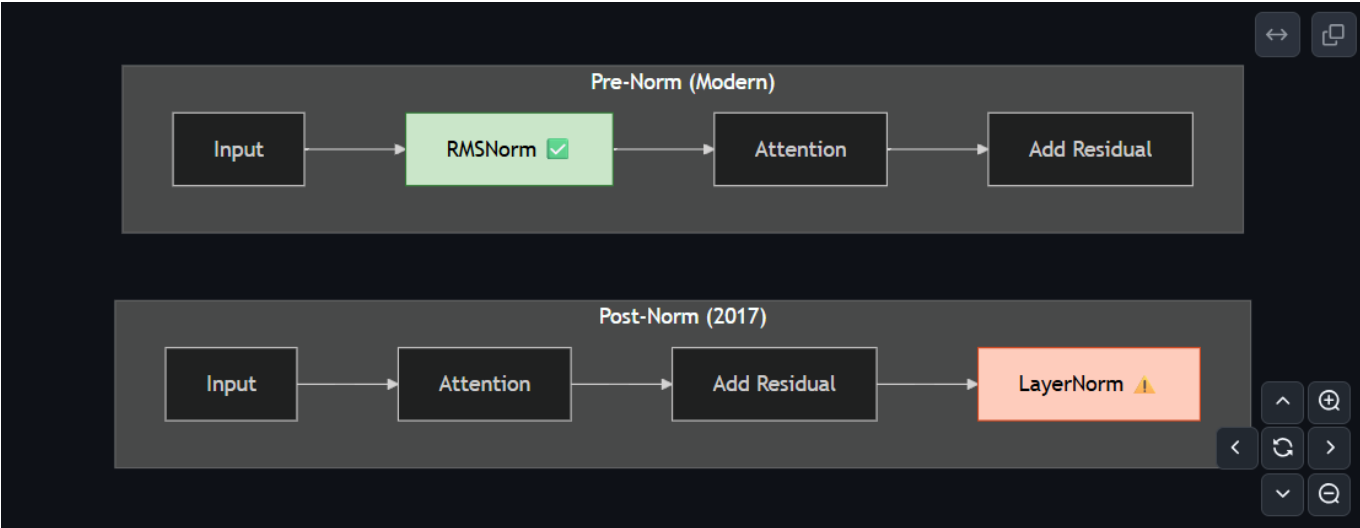
Component	Classic (2017-2019)	Modern (2023-2025)	The "Why"
Normalization	LayerNorm (Post-Norm)	RMSNorm (Pre-Norm)	Removes mean/bias calc; Pre-Norm sends raw signal through residual path for stable gradients at 100+ layers.
Feed-Forward	ReLU FFN	SwiGLU	Smooth, learned gating replaces blunt zero-cutoff. The network decides <i>how much</i> signal to pass, not just <i>whether</i> .
Position Encoding	Sinusoidal (Absolute)	RoPE (Rotary)	Encodes <i>relative</i> distance ("how far apart are these words?") instead of absolute slot numbers. Generalizes to unseen sequence lengths.
Attention	Multi-Head (MHA)	Grouped Query (GQA)	Groups of Query heads share KV pairs → 4-8x KV Cache reduction → longer contexts, more concurrent users.

[!TIP] **The Mental Model:** Think of these not as "new ideas" but as **engineering patches** applied to a fundamentally correct design. The core insight of Self-Attention was right all along; the packaging just needed to scale.

3 Normalization: RMSNorm & Pre-Norm

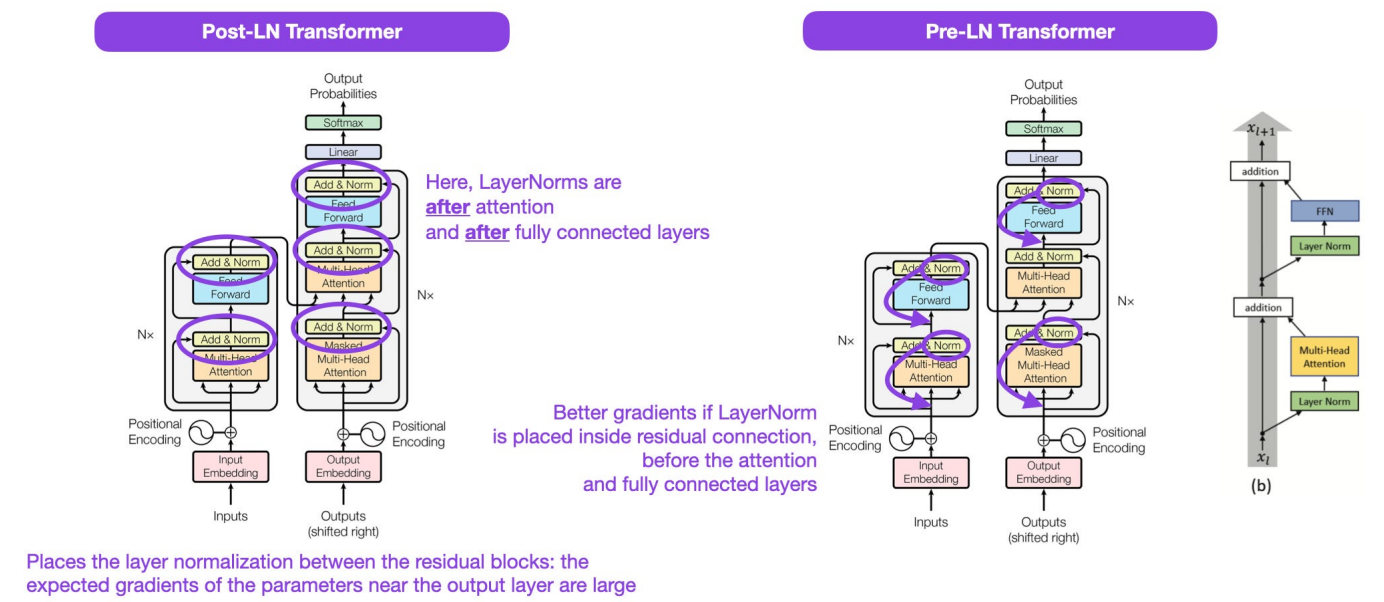
When stacking 30, 80, or 120 layers deep, the variance of your signal can easily explode to infinity or vanish to zero. Normalization acts as a **guardrail** keeping the numbers in a healthy range at every layer.

🔊 Pre-Norm vs. Post-Norm: Where You Normalize Changes Everything



- **Post-Norm (Original):** Normalizing *after* the sublayer means the residual connection has to pass through the normalization before reaching the next layer. At 30+ layers, gradients struggle to flow cleanly backward through all those normalization barriers. It's like passing a message through 30 translators — each one subtly distorts it.

- **Pre-Norm (Modern):** We normalize the signal *before* feeding it into the sublayer. The crucial trick: **the residual connection now carries the raw, unadulterated signal** directly from layer 1 to layer 120. Think of it as a **highway express lane** that bypasses all the toll booths (normalizations). The tolls are still paid, but only on the *side road* (the sublayer path). This dramatically stabilizes training for ultra-deep models.



[!NOTE] **Key difference:** RMSNorm maintains a **non-zero mean** (unlike LayerNorm which centers at zero). Empirically, this doesn't hurt training stability but saves significant computation.

📖 The Implementation (Just 6 Lines of Real Code)

```
class RMSNorm(nn.Module):
    """Root Mean Square Layer Normalization – used in LLaMA, Mistral, Gemma."""
    def __init__(self, dim, eps=1e-6):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(dim)) # Learnable scale (γ)
        # Note: NO bias parameter. That's the whole point.

    def forward(self, x):
        # Step 1: Square → Mean → Root (the "RMS" in RMSNorm)
        rms = torch.sqrt(x.pow(2).mean(-1, keepdim=True) + self.eps)
        # Step 2: Normalize by dividing, then scale by learned weight
        return (x / rms) * self.weight
```

[!TIP] **The Analogy:** Imagine you're grading exams. LayerNorm first calculates the class average, subtracts it from every score, then divides by the spread. RMSNorm just divides every score by the "overall magnitude" of the class. Both produce comparable rankings, but RMSNorm does it with fewer operations — crucial when you're grading **billions** of exams per second.

4 Activation: SwiGLU & Learned Gating

The classic FFN just used $\text{ReLU}(x) = \max(0, x)$. If a number was negative, it was killed to zero. Simple, fast, but brutally blunt — like a bouncer who either lets you in 100% or throws you out entirely.

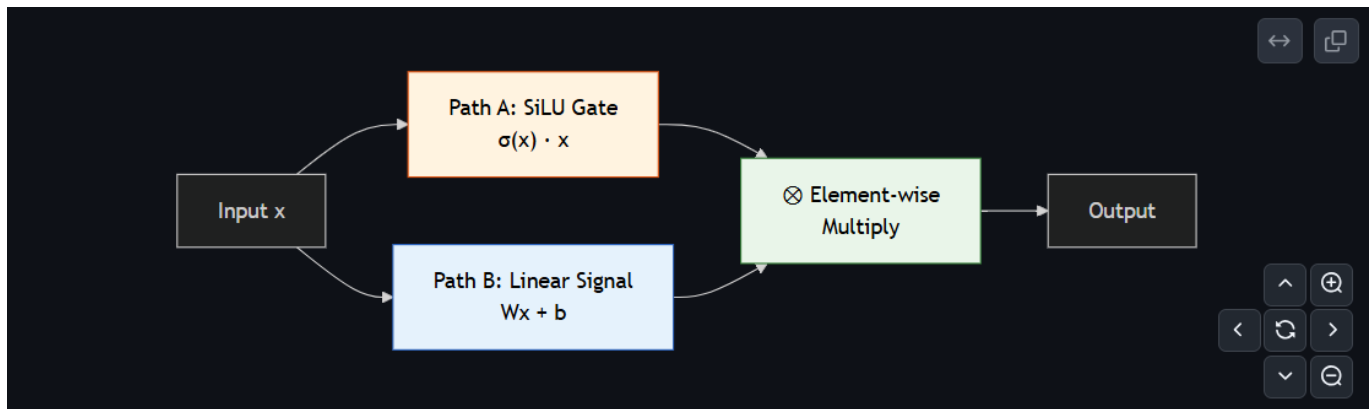
📖 The Problem with ReLU

```
ReLU(x):
Input:  [-2.1, 0.5, -0.3, 1.8, -4.0]
Output: [ 0.0, 0.5,  0.0, 1.8,  0.0] ← 60% of information DESTROYED
```

ReLU kills all negative activations. This means half your neurons contribute *nothing* at any given time. This also causes the **"Dying ReLU" problem**: if a neuron's output is consistently negative, its gradient is permanently zero and it never learns again.

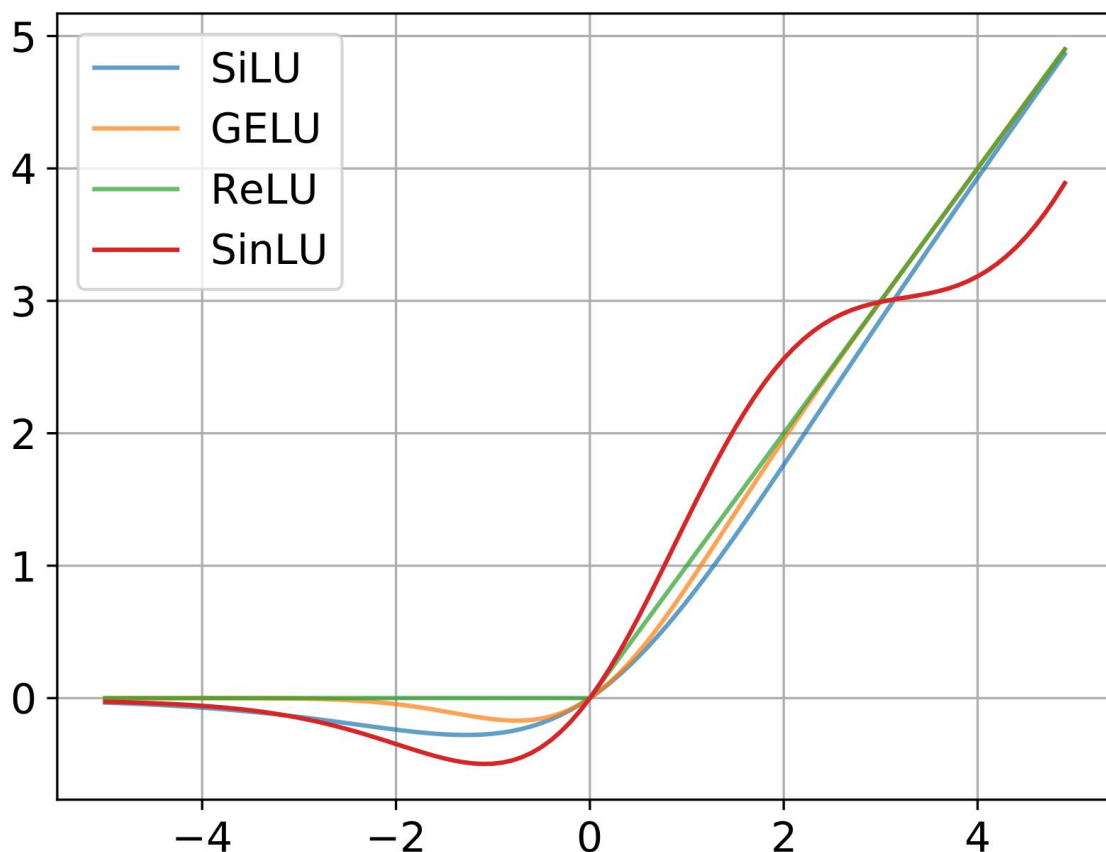
🧠 SwiGLU: A Smarter Gatekeeper

SwiGLU (Swish Gated Linear Unit) replaces the blunt bouncer with a **sophisticated security system** using two paths:



1. **Path A — The Gate:** The input passes through $\text{SiLU}(x) = x \cdot \text{sigmoid}(x)$. This produces a smooth value between roughly -0.3 and $+\infty$, acting as a **probability dial** for how much signal should pass.
2. **Path B — The Signal:** A separate linear transformation of the input carries the raw information.
3. **The Merge:** The gate and the signal are multiplied element-wise. The gate *controls* how much of each feature dimension flows forward.

ReLU vs. SiLU: The Activation Curve



Comparison of activation functions: ReLU has a hard kink at 0 (gradient is exactly 0 for $x < 0$, causing "dying neurons"). SiLU/Swish provides a smooth, non-monotonic curve — small negative values get a tiny negative output, so gradients always flow.

[!IMPORTANT] **Why SwiGLU wins at scale:**

1. **No dead neurons:** The smooth curve means gradients always flow, even for negative inputs.
2. **Learned gating:** The network dynamically decides *how much* of each feature to let through, based on context. ReLU can't do this — it's a fixed threshold.
3. **Richer representations:** Two learned projections (gate + signal) give the FFN more expressive power than a single projection through ReLU.

 SwiGLU in Code

```
class SwiGLU_FFN(nn.Module):
    """The modern Feed-Forward Network used in LLaMA, Mistral, etc."""
    def __init__(self, dim, hidden_dim):
        super().__init__()
        self.w_gate = nn.Linear(dim, hidden_dim, bias=False) # Gate path
        self.w_up   = nn.Linear(dim, hidden_dim, bias=False) # Signal path
        self.w_down = nn.Linear(hidden_dim, dim, bias=False) # Project back

    def forward(self, x):
        gate = F.silu(self.w_gate(x)) # Path A: SiLU activation on gate
        signal = self.w_up(x)         # Path B: Linear signal
        return self.w_down(gate * signal) # Merge → Project down
```

[!NOTE] **Trade-off:** SwiGLU uses **3 weight matrices** instead of ReLU's 2. The hidden dimension is typically reduced slightly (e.g., $\frac{2}{3} \times 4d$ instead of $4d$) to keep the total parameter count similar. The result is a *better* FFN at roughly the same cost.

5 Positional Encoding: RoPE

Transformers have no inherent sense of word order. Feed it "the cat sat on the mat" or "mat the on sat cat the" — the attention mechanism produces the **exact same output** (it's permutation-invariant). We must explicitly inject position.

⊗ The Problem with Sinusoidal Positions (2017)

The original paper *added* a fixed sinusoidal wave pattern to the token embeddings:

```
Position 0: [sin(0/10000^0), cos(0/10000^0), sin(0/10000^2/d), ...]
Position 1: [sin(1/10000^0), cos(1/10000^0), sin(1/10000^2/d), ...]
...
```

This has two fundamental limitations:

Problem	Explanation	Consequence
Absolute Encoding	Each position gets a fixed tag: "I am token #47."	The model memorizes <i>positions</i> , not <i>relationships</i> . Token #47 in a 100-word sentence and token #47 in a 10,000-word document are treated identically.

Problem	Explanation	Consequence
No Relative Distance	The encoding doesn't naturally capture "how far apart are two tokens?"	In language, the relationship between an adjective and its noun matters far more than either's absolute position in a book. "The big dog " has the same adjective-noun distance whether it appears at position 5 or position 5,000.
Fixed Context Length	Trained on sequences of length L; struggles to extrapolate to L + 1.	Can't handle longer prompts than it was trained on without significant degradation.

 Enter RoPE (Rotary Position Embedding)

Instead of *adding* a position vector to the embedding, RoPE **rotates** the Query and Key vectors in multi-dimensional space based on their position in the sequence. This is a fundamentally different approach.

 The Clock Analogy

Imagine each Q/K vector as a hand on a clock:

Token at position 3:  (rotated to 3 o'clock)

Token at position 5:  (rotated to 5 o'clock)

Angle between them: 2 hours ← RELATIVE DISTANCE

Shift entire sentence forward by 4 positions:

Token at position 7:  (rotated to 7 o'clock)

Token at position 9:  (rotated to 9 o'clock)

Angle between them: STILL 2 hours 

When Attention computes the dot product $Q_m \cdot K_n$, the rotation is designed so that the result **depends only on the angle between the vectors** — i.e., the **relative distance** $(m - n)$. Absolute positions vanish from the equation entirely.

 RoPE Mechanics: How It Actually Works

1. **Dimension Pairing:** For a vector of dimension d , RoPE groups dimensions into pairs: $(x_1, x_2), (x_3, x_4), \dots, (x_{d-1}, x_d)$. Each pair lives on a unique 2D plane.

2. **2D Rotation:** Each pair is rotated by an angle $\theta_i \cdot m$, where m is the token's position index and θ_i is a frequency specific to that dimension pair.

$$\begin{pmatrix} x'_{2i} & x'_{2i+1} \end{pmatrix} = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix} \begin{pmatrix} x_{2i} & x_{2i+1} \end{pmatrix}$$

3. **Frequency Scaling (Multi-Resolution):** This is the elegant part: $\theta_i = 10000^{-2i/d}$.

- Low-index pairs (small i): θ is large → rotates **fast** → captures **nearby** token relationships (syntax, grammar).
- High-index pairs (large i): θ is small → rotates **slowly** → captures **distant** relationships (topic, narrative arc).

[!TIP] **Multi-Resolution Analogy:** Think of a telescope with multiple lenses. Low dimensions are the **zoom lens** (sharp detail on nearby words), high dimensions are the **wide-angle lens** (broad view of

the whole paragraph). Each pair of dimensions operates at a different "zoom level," and together they capture relationships at every scale simultaneously.

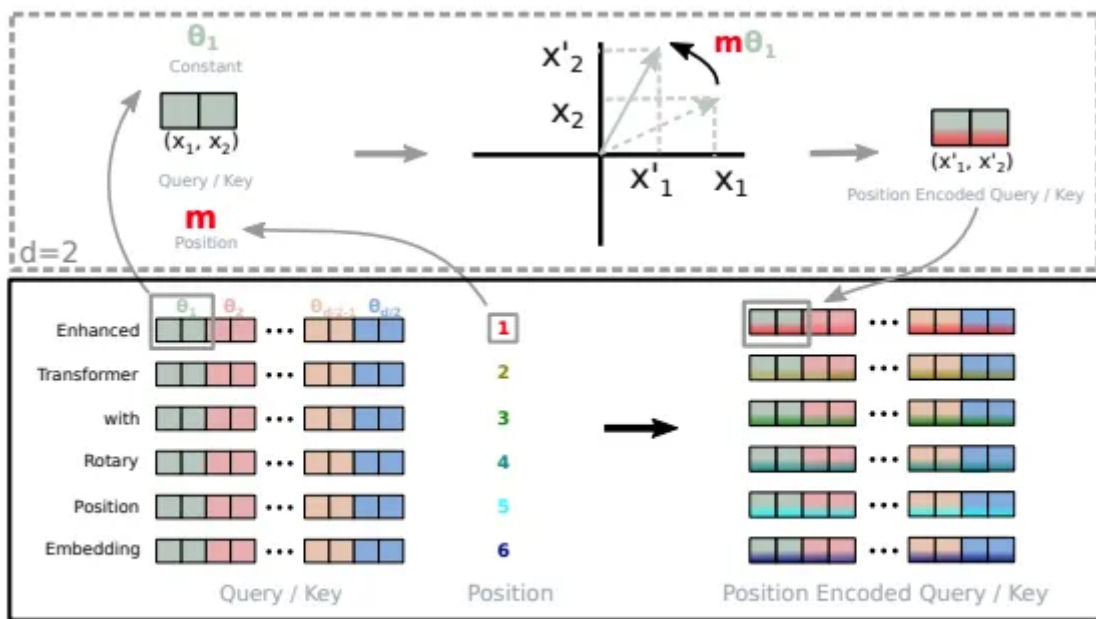


Figure 1: Implementation of Rotary Position Embedding(RoPE).

Visualizing RoPE: Each pair of dimensions (x_1, x_2) is treated as a 2D coordinate and rotated by an angle proportional to the token's position index m . The rotation ensures the dot product only cares about the relative angle between tokens.

🔑 Why RoPE Enables Longer Contexts

Because position is encoded as a *rotation angle* rather than a fixed *added vector*, the model can theoretically extrapolate to sequence lengths it never saw during training. Techniques like **YaRN** (Yet another RoPE extension) and **NTK-aware scaling** adjust the rotation frequencies to extend context from 4K → 128K → 1M+ tokens.

6 Attention Deep Dive: MHA → GQA

To understand why the attention mechanism had to change, we need to think about what happens during **inference** — specifically, autoregressive text generation (producing one token at a time).

🏠 The KV Cache: Why Standard Attention Hits a Memory Wall

When generating token #501, the model must attend to all 500 previous tokens. Naively recomputing K and V for every past token at every step is computationally impossible. The solution: **cache them**.

⚡ Autoregressive Generation
– Token no. 501

New Token no.
501
Compute Q, K, V

KV Cache
K1 to K500, V1
to V500

Attention
 $Q_{501} \times K_1$ to
K501 transposed

Output Token

$$\text{KV Cache Size} = \underbrace{2}_{\text{K and V}} \times \underbrace{H}_{\text{heads}} \times \underbrace{S_{\text{past}}}_{\text{seq length}} \times \underbrace{d_{\text{head}}}_{\text{head dim}} \times \underbrace{\text{bytes}}_{\text{precision}}$$

 The Concrete Math (32-Layer Model)

Parameter	Value
Layers	32
Heads per layer (H)	32
Head dimension (d_{head})	128
Sequence length	8,192 tokens
Precision	float16 (2 bytes)

Per Layer = $2 \times 32 \times 8192 \times 128 \times 2 = 128 \text{ MB}$
Total (32 layers) = $128 \times 32 = 4 \text{ GB}$

That's **4 GB of GPU memory** just to store the KV cache for a single user with 8K context. For **128K context windows**, this balloons to **~64 GB** — exceeding the VRAM of most GPUs before the model even starts “thinking.”

 Time Complexity of Attention

The two dominant operations in the attention forward pass:

Operation	Shape Computation	Time Complexity
Dot Product QK^T	$[B, H, S, d_h] \times [B, H, d_{kv}, S]$	$O(B \cdot H \cdot S^2 \cdot d_{\text{head}})$
Weighted Sum Score $\cdot V$	$[B, H, S, S] \times [B, H, S, d_h]$	$O(B \cdot H \cdot S^2 \cdot d_{\text{head}})$
Softmax	Element-wise on $[B, H, S, S]$	$O(B \cdot H \cdot S^2)$

 Note

Why the $\frac{1}{\sqrt{d_{\text{head}}}}$ Scaling Factor?

If Q and K have independent components with mean 0 and variance 1, their dot product has **mean 0 but variance d_{head}** . As d_{head} grows (typically 64–128), the raw dot product values become enormous.

- **Softmax Saturation:** Huge values push softmax into its flat tails where $\frac{\partial \text{softmax}}{\partial x} \approx 0$. Gradients vanish during backprop.
- **Numerical Overflow:** The exponentials inside softmax (e^x) overflow to `inf` for large x .

Dividing by $\sqrt{d_{\text{head}}}$ rescales the variance back to 1, keeping softmax in its sensitive, gradient-rich region.

 The Spectrum: MHA → GQA → MQA

The solution to the KV Cache memory wall is **sharing** Key/Value heads across groups of Query heads.

	Query Heads	Key Heads	Value Heads
MHA (H=8)	Q ₁ Q ₂ Q ₃ Q ₄ Q ₅ Q ₆ Q ₇ Q ₈	K ₁ K ₂ K ₃ K ₄ K ₅ K ₆ K ₇ K ₈	V ₁ V ₂ V ₃ V ₄ V ₅ V ₆ V ₇ V ₈
GQA (G=2)	Q ₁ Q ₂ Q ₃ Q ₄ Q ₅ Q ₆ Q ₇ Q ₈	K ₁ ————— K ₂ —————	V ₁ ————— V ₂ —————
MQA (G=1)	Q ₁ Q ₂ Q ₃ Q ₄ Q ₅ Q ₆ Q ₇ Q ₈	K ₁ —————	V ₁ —————

Method	Q Heads	K/V Heads	KV Cache Reduction	Quality Impact
MHA (Multi-Head)	H	H	1x (baseline)	Best quality
GQA (Grouped-Query)	H	G (where $1 < G < H$)	H / G reduction	Near-MHA quality ✅
MQA (Multi-Query)	H	1	$H \times$ reduction	Measurable quality drop ⚠️

The Interpolation Property:

- Set $G = H \rightarrow$ GQA becomes MHA (every head has its own K/V).
- Set $G = 1 \rightarrow$ GQA becomes MQA (all heads share one K/V).

🔧 GQA Step-by-Step Process

- Project** the input X into Q, K, V via linear transformations.
- Split Q** into H total query heads and **group** them into G groups (each group has H / G query heads).
- Split K and V** into only G heads (one K/V pair per group).
- Within each group:** All H / G query heads attend using the **same** shared K and V .
- Concatenate** all head outputs and project through a final linear layer.

[!IMPORTANT] **The Real-World Result:** GQA achieves a **4x to 8x reduction** in KV cache memory with **almost zero quality degradation**. This is not just theoretical — the improvement comes from reduced **memory bandwidth demands** and improved **cache locality** on real GPU hardware. LLaMA 2 70B, Mistral 7B, and Gemma all use GQA.

🔭 Modern Spotlight: DeepSeek's MLA (Multi-Head Latent Attention)

While GQA is the 2024 standard, **DeepSeek-V3** recently introduced **MLA** to push the "Memory Wall" even further.

- The Idea:** Instead of just sharing heads, MLA uses **Low-Rank Compression**. It compresses the entire KV cache into a tiny "latent vector" and only decompresses it *inside* the attention head.
- The Result:** MLA achieves a **KV Cache reduction of up to 90%** compared to standard MHA, allowing models to handle millions of tokens with minimal VRAM.

🔗 Uptraining: Converting MHA $\rightarrow$ GQA Cheaply

You don't need to train a GQA model from scratch. The GQA paper (Ainslie et al., 2023) showed you can take an existing MHA checkpoint and **uptrain** it with GQA by:

- Averaging (mean-pooling) the K and V projection weights within each group.
- Training for only **5-10% of the original compute** to recover quality.

This makes GQA adoption extremely cost-effective.

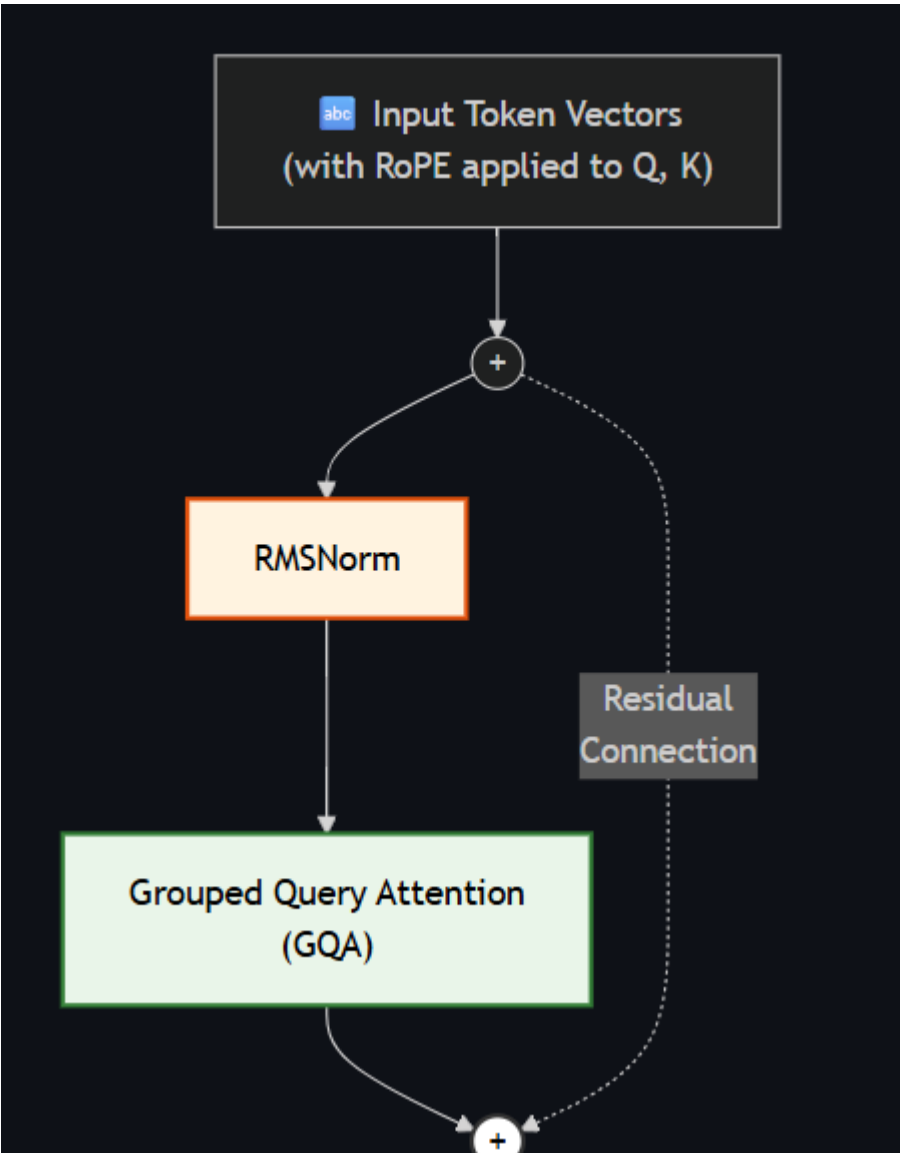
📖 Other Optimization Methods (Beyond Attention)

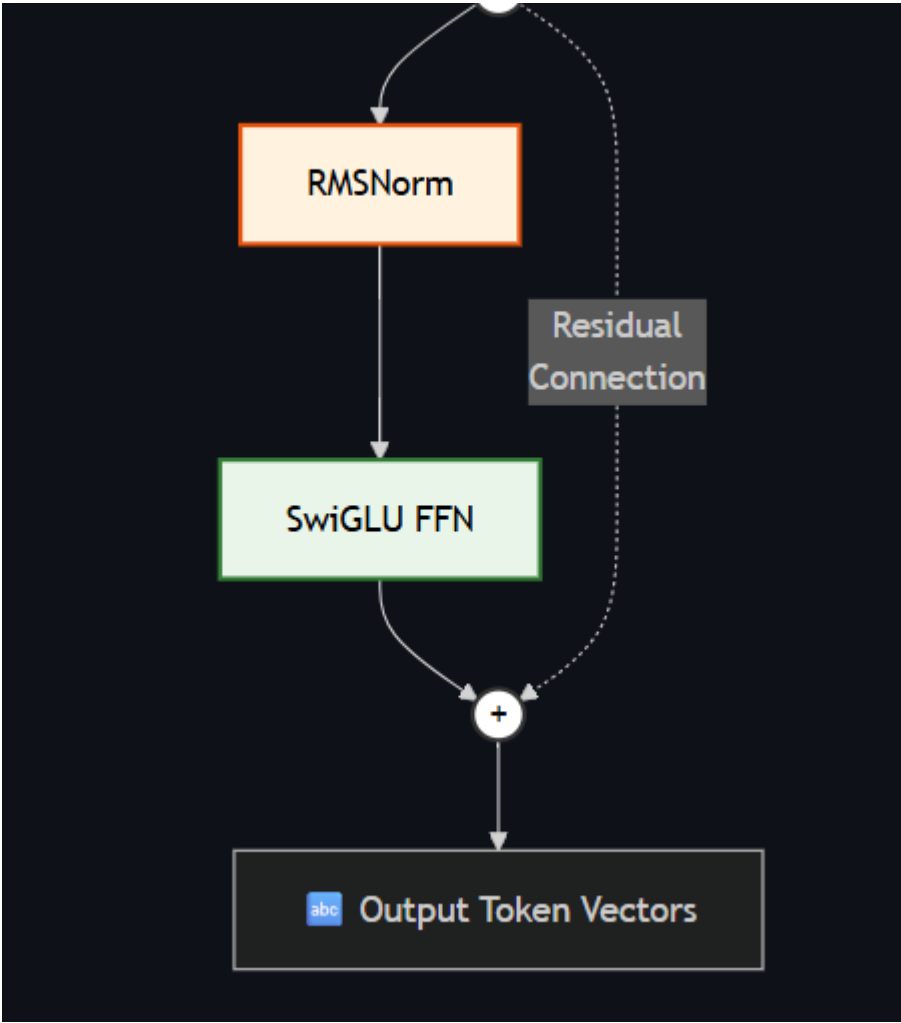
The attention mechanism is only one dimension of optimization. Other important techniques include:

Method	What It Does
Flash Attention (Dao, 2022)	Fuses attention kernels to minimize GPU memory reads/writes. Exact same math, dramatically faster.
LoRA (Hu, 2021)	Efficient fine-tuning by adding small low-rank matrices instead of updating all weights.
Quantization (LLM.int8, GPTQ)	Reduces weight precision (fp16 → int8 → int4) to shrink model size and speed up inference.
Speculative Decoding (Chen, 2023)	Uses a small "draft" model to propose tokens in parallel, verified by the large model.

7 The Modern Transformer Block

When we assemble all four upgrades, we get the architecture powering virtually every state-of-the-art open-weight LLM in 2024+.





🔗 Reading the Block: What Happens to a Token

Step	Operation	What's Happening
1	RoPE	Q and K vectors are <i>rotated</i> based on position → relative distance is baked in.
2	RMSNorm	Signal is normalized (Pre-Norm) before entering attention → stable gradients.
3	GQA	Grouped attention with shared K/V heads → efficient memory usage.
4	Add (Residual)	Raw input is added back → gradient highway stays intact.
5	RMSNorm	Signal normalized again before FFN.
6	SwiGLU FFN	Two-path gated transformation → richer, smoother feature extraction.
7	Add (Residual)	Raw signal added back again → unimpeded gradient flow.

[!TIP] **How to read any modern LLM architecture paper:** Every decoder block is just this pattern repeated N times (N = 32 for 7B models, 80 for 70B models, 120+ for 400B models). The differences between LLaMA, Mistral, Gemma, etc. are mostly in the *hyperparameters* (number of layers, hidden dim, number of GQA groups), not the architecture.

📊 The Final Comparison: 2017 vs. 2024

Feature	Classic Transformer (2017)	Modern LLM Stack (2024+)
---------	----------------------------	--------------------------

Feature	Classic Transformer (2017)	Modern LLM Stack (2024+)
Normalization	LayerNorm (Post-Norm)	RMSNorm (Pre-Norm)
Position	Absolute Sinusoidal	Rotary (RoPE)
Attention	Multi-Head (MHA)	Grouped-Query (GQA)
Activation	ReLU	SwiGLU
Architecture	Encoder-Decoder	Decoder-Only
Primary Focus	"Can this even work?"	Stability, Efficiency, & Scale

[!NOTE] **The Takeaway:** The fundamental idea of the Transformer — Self-Attention — has survived entirely intact. The upgrades of the past 7 years haven't been about changing *how* it thinks, but surgically optimizing *how efficiently* it can route information, maintain gradient stability, and manage memory at massive scales. The 2017 paper got the core idea right; the 2024 stack makes it *practical* at 400B+ parameters.



Quick Revision Card

The entire lecture on one page.

 Quick Revision Card

The entire lecture on one page.

Upgrade	Classic	Modern	One-Sentence Why
Norm	LayerNorm (Post)	RMSNorm (Pre)	Remove mean/bias math; put norm <i>before</i> sublayer so residuals carry raw signal.
FFN	ReLU	SwiGLU	Smooth learned gate > blunt cutoff. Two paths let the network <i>decide</i> how much signal passes.
Position	Sinusoidal (add)	RoPE (rotate)	Rotate Q/K by position angle → dot product depends on relative distance, not absolute slot.
Attention	MHA	GQA	Share K/V across groups of Q heads → 4-8x less KV cache memory at near-zero quality loss.

Key Formulas:

- RMSNorm:** $\text{RMS}(x) = \sqrt{\frac{1}{d} \sum x_i^2}$, then $\hat{x} = \frac{x}{\text{RMS}} \cdot \gamma$
- SiLU (Swish):** $\text{SiLU}(x) = x \cdot \sigma(x)$
- RoPE Rotation:** $x'_{2i} = x_{2i} \cos(m\theta_i) - x_{2i+1} \sin(m\theta_i)$, $x'_{2i+1} = x_{2i} \sin(m\theta_i) + x_{2i+1} \cos(m\theta_i)$
- KV Cache:** Size = $2 \times H \times S \times d_h \times \text{bytes per layer}$
- Scaled Dot Product:** $\text{Attention} = \text{softmax!} \left(\frac{QK^T}{\sqrt{d_{\text{head}}}} \right) V$

Key Numbers (LLaMA 2 — 7B):

Param	Value
Layers	32
Hidden Dim (d)	4096
Heads (H)	32
GQA Groups (G)	4 (in 70B variant)
Head Dim (d_h)	128
FFN Hidden	$\frac{2}{3} \times 4 \times 4096 = 11008$
Vocab Size	32,000